

PYTHON PROGRAMMING

Comprehensions in Python

List · Dictionary · Nested · Performance & Practical Use

Class Notes | 2026

Contents

Part 1 Introduction to Comprehensions

Part 2 List Comprehensions

Part 3 Conditional Filtering

Part 4 Dictionary Comprehensions

Part 5 Nested Comprehensions

Part 6 Performance & Benefits

Part 7 Practical Activities

Part 8 Interview Questions

PART 1

Introduction to Comprehensions

What they are and why Python introduced them

What is a Comprehension?

A **comprehension** is a concise, expressive way to build collections (lists, dictionaries, sets) from existing iterables in a single readable line. Instead of writing a multi-line for-loop with an append call, you describe what you want — and Python creates it.

■ **Key Insight:** *Comprehensions combine the loop, the expression, and an optional condition into one elegant statement.*

Why Were Comprehensions Introduced?

Reason	Details
Readability	Mirrors mathematical set-builder notation: $\{ x \mid x \in S, x > 0 \}$
Conciseness	Reduces 4–6 lines of loop code to a single expressive line
Performance	CPython optimises list comprehensions – faster than equivalent for-loops
Pythonic style	Favoured in the Python community; used widely in real codebases

Traditional Loop vs. Comprehension — Side by Side

Traditional for-loop approach:

```
# Build a list of squares using a for-loop

squares = []

for x in range(1, 6):
    squares.append(x * x)

print(squares) # [1, 4, 9, 16, 25]
```

Comprehension approach:

```
# Same result – one line!
```

```
squares = [x * x for x in range(1, 6)]  
print(squares) # [1, 4, 9, 16, 25]
```

PART 2

List Comprehensions

Creating lists the Pythonic way

Core Syntax

```
[ expression for item in iterable ]
```

Reading tip: Read it as: *For every item in the iterable, evaluate the expression and collect the result.*

How It Works — Step by Step

- 1 Python iterates over every element in the iterable.
- 2 For each element, it evaluates the expression.
- 3 The results are collected into a new list automatically.

Examples from Class

Example 1 — Adding 2 to each number:

```
list_nums = [1, 2, 3, 4, 5]

# Traditional approach

result = []

for i in list_nums:
    result.append(i + 2)

# List comprehension

list_num = [i + 2 for i in list_nums]

print(list_num) # [3, 4, 5, 6, 7]
```

Example 2 — Squares of numbers:

```
list_nums = [1, 2, 3, 4, 5]
```

```
squares_num = [x * x for x in list_nums]

print(squares_num) # [1, 4, 9, 16, 25]
```

Example 3 — Uppercase strings:

```
words = ["hello", "world", "python"]

upper = [w.upper() for w in words]

print(upper) # ["HELLO", "WORLD", "PYTHON"]
```

PART 3

Conditional Filtering

Adding if-conditions to comprehensions

Syntax with Condition (Filter)

```
[ expression for item in iterable if condition ]
```

Note: The *if* at the end acts as a **filter** — only items satisfying the condition are included in the result.

Examples from Class

Example 1 — Even numbers from 1 to 10:

```
# Traditional approach

list_num = []

for i in range(1, 11):

    if i % 2 == 0:

        list_num.append(i)

# Comprehension with condition

even_list_num = [i for i in range(1, 11) if i % 2 == 0]

print(even_list_num) # [2, 4, 6, 8, 10]
```

Example 2 — Names starting with 'A' or 'J':

```
names = ["JRonak", "Khushboo", "ATanishq"]

# Using startswith()

result = [name for name in names

if name.startswith("A") or name.startswith("J")]

# Using indexing — equivalent approach

result = [name for name in names
```

```
if name[0] == "A" or name[0] == "J"]

print(result) # ["JRonak", "ATanishq"]
```

Ternary (if-else) Inside Comprehension

```
[ val_if_true if condition else val_if_false for item in iterable ]
```

```
# Label numbers as 'even' or 'odd'

labels = ["even" if x % 2 == 0 else "odd" for x in range(1, 6)]

print(labels) # ["odd", "even", "odd", "even", "odd"]
```

PART 4

Dictionary Comprehensions

Building key-value stores concisely

Syntax

```
{ key : value for item in iterable }
```

Remember: Dictionaries use **curly braces { }** and require a **key : value** pair as the expression — unlike lists which use square brackets.

Examples from Class

Example 1 — Squares dictionary:

```
# Traditional approach

squares_dict = {}

for i in range(1, 6):

    squares_dict[i] = i * i

# Dictionary comprehension

squares_dict = {i: i*i for i in range(1, 6)}

print(squares_dict) # {1:1, 2:4, 3:9, 4:16, 5:25}
```

Example 2 — 10% salary raise for all employees:

```
employees_dict = {

    "John": 55000,

    "Jane": 65000,

}

# Give every employee a 10% raise

updated = {name: salary * 1.1

for name, salary in employees_dict.items()}
```

```
print(updated)

# {'John': 60500.0, 'Jane': 71500.0}
```

Example 3 — Filtering inside a dict comprehension:

```
# Only include employees earning more than 58,000

high_earners = {name: salary
for name, salary in employees_dict.items()
if salary > 58000}

print(high_earners) # {'Jane': 65000}
```

PART 5

Nested Comprehensions

Working with matrices and multi-dimensional data

Syntax

```
[ expression for item1 in iterable1 for item2 in iterable2 ]
```

Reading rule: Read nested comprehensions **left to right**, exactly as you would write the nested for-loops — outer loop first, inner loop next.

Example 1 — Flatten a matrix and filter even values:

```
matrix = [
    [1, 2, 3],
    [4, 5],
    [7, 8, 9]
]

# Traditional nested loops
for row in matrix:
    if len(row) == 3:
        for cell in row:
            if cell % 2 == 0:
                print(cell)

# Nested comprehension – same logic
result = [cell for row in matrix for cell in row if cell % 2 == 0]
print(result) # [2, 8]
```

Example 2 — Multiplication table (5×5):

```
table = [[i * j for j in range(1, 6)] for i in range(1, 6)]
print(table)
```

```
# [[1,2,3,4,5], [2,4,6,8,10], [3,6,9,12,15],  
# [4,8,12,16,20], [5,10,15,20,25]]
```

Example 3 — Pairs where product is even:

```
result = [  
    (i, j)  
    for i in range(1, 4)  
    for j in range(i, 4)  
    if (i * j) % 2 == 0  
]  
  
print(result) # [(1, 2), (2, 2), (2, 3), (3, 2)] (approx)
```

Example 4 — Rows of length 3 only:

```
even_row = [cell for row in matrix if len(row) == 3 for cell in row]  
  
print(even_row) # [1, 2, 3, 7, 8, 9]
```

PART 6

Performance & Benefits

Why comprehensions are faster and more readable

Speed Comparison

CPython (the standard Python interpreter) applies **bytecode-level optimisations** specifically to list comprehensions that are not available for equivalent for-loops. Benchmark results typically show comprehensions running **10–35% faster**.

```
import time

# For-loop timing
start = time.time()
result = []
for x in range(1_000_000):
    result.append(x * x)
print(f'Loop: {time.time() - start:.4f}s')

# Comprehension timing
start = time.time()
result = [x * x for x in range(1_000_000)]
print(f'Comprehension: {time.time() - start:.4f}s')

# Comprehension is typically faster!
```

Why Is It Faster?

Factor	Explanation
No attribute lookup	For-loops call <code>list.append()</code> on each iteration – that's an attribute lookup + call. Comprehensions bypass this.
Optimised bytecode	CPython uses <code>LIST_APPEND</code> opcode internally which is more efficient than <code>CALL_FUNCTION</code> .
Pre-allocated memory	The interpreter can sometimes allocate the result list more efficiently.

When NOT to Use Comprehensions

- When the expression is very complex — prefer a named function for clarity.
- When you need side effects (e.g. printing inside the loop) — use a plain loop.
- When the comprehension spans more than 2–3 lines — break it into a loop.
- For very large datasets where memory matters — consider a generator expression instead.

Generator Tip: Replace `[ ]` with `( )` to create a **generator expression** — it is lazy (computes values on demand) and uses almost no memory. Great for large data pipelines.

PART 7

Practical Activities

Solved exercises from the class session

Activity 1 — GST Calculator

Given a list of product prices, calculate the price including 18% GST for each product.

```
prices = [100, 250, 80, 430, 199]

# Price after adding 18% GST

gst_prices = [price * 1.18 for price in prices]

print(gst_prices)

# [118.0, 295.0, 94.4, 507.4, 234.82]
```

Activity 2 — Convert Strings to Uppercase

Transform a list of lowercase city names into uppercase.

```
cities = ["mumbai", "delhi", "bangalore", "hyderabad"]

upper_cities = [city.upper() for city in cities]

print(upper_cities)

# ["MUMBAI", "DELHI", "BANGALORE", "HYDERABAD"]
```

Activity 3 — Squares Dictionary

Build a dictionary mapping numbers 1–5 to their squares.

```
squares = {i: i*i for i in range(1, 6)}

print(squares)

# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Activity 4 — Filter Names

From a list of names, keep only those starting with 'A' or 'J'.

```
names = ["JRonak", "Khushboo", "ATanishq", "Mehul", "Ananya"]  
result = [n for n in names if n[0] in ("A", "J")]  
print(result) # ["JRonak", "ATanishq", "Ananya"]
```

Activity 5 — Cubes List

Generate a list of cubes for numbers 1 through 10.

```
cubes = [x**3 for x in range(1, 11)]  
print(cubes)  
# [1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]
```

PART 8

Interview Questions

Common questions asked in Python interviews

Q1. What is a list comprehension? How is it different from a for-loop?

A list comprehension is a concise syntax to build a list by combining a loop and an optional condition into a single expression. Unlike a regular for-loop (which is general-purpose and runs statements), a comprehension always produces a list as output and is typically 10–35% faster due to CPython optimisations.

Q2. What is the syntax of a list comprehension with filtering?

```
[expression for item in iterable if condition]
```

Example: `[x for x in range(10) if x % 2 == 0]` → `[0, 2, 4, 6, 8]`

Q3. What is a dictionary comprehension? Give an example.

A dictionary comprehension builds a dict using `{key: value for item in iterable}`.

Example: `{x: x**2 for x in range(1, 4)}` → `{1:1, 2:4, 3:9}`

Q4. What is the difference between a list comprehension and a generator expression?

List comprehension `[ ]` builds the entire list in memory immediately. Generator expression `( )` is lazy — it yields one value at a time and uses far less memory. Use generators for large datasets: `sum(x*x for x in range(1_000_000))`

Q5. Can you write an if-else inside a list comprehension?

Yes — the ternary goes before the for:

```
["even" if x%2==0 else "odd" for x in range(5)]
```

Note: the filtering if (after for) cannot have an else; only the expression-level ternary can.

Q6. How does a nested comprehension work? When should you avoid it?

Nested comprehensions use multiple for clauses:

```
[x+y for x in [1,2] for y in [10,20]] → [11, 21, 12, 22]
```

Avoid them when readability suffers — prefer explicit nested loops for complex logic.

Q7. What happens if duplicate keys appear in a dict comprehension?

The last value wins — same as a regular dict. Example:

```
{k: v for k, v in [('a',1), ('a',2)]} → {'a': 2}
```

Q8. Write a one-liner to transpose a 3×3 matrix using comprehensions.

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]  
transposed = [[row[i] for row in matrix] for i in range(3)]  
# [[1,4,7],[2,5,8],[3,6,9]]
```

Pro tip for interviews: Always mention that comprehensions are more Pythonic, faster due to bytecode optimisations, and that generator expressions are the memory-efficient alternative for large data.